Useful Queries, Commands, and Tidbits

- Queries
  - Event Related
  - Aligned Dynamic Apps/Collectors/Powerpacks etc
    - Cleaning up unused PPKs and DAs
  - MariaDB Internals
  - Everything else
  - Collector related
  - Execution Environment related
  - Scheduler Related
  - General Usecases
    - Collecting info on Devices
    - Searching MetaData (PowerPack and DA code, finding tables and columns in SQL, etc)
    - Miscellaneous
- Less Useful
  - Queries used by UI
    - Classic
- Other Userful Query Resources:
- Useful Commands (Non-sql)
- Tidbits
  - Packet Captures on ESXI Hosts
  - Pruner and Daily Maintenance

Queries

Event Related

- Find all active events with 10 or more occurances and list the count of active events, the event policy ID, and the event policy name.

```
SELECT COUNT(0), ea.etype, pe.ename
FROM master_events.events_active AS ea
JOIN master.policies_events AS pe
  ON ea.etype = pe.id
WHERE ea.estate = 0
GROUP BY ea.etype
HAVING COUNT(0) > 9
ORDER BY COUNT(0) DESC;
```

- Given an event policy ID (ETYPE) give a list of all collectors with devices that have that event active and a count of the number of devices.

```
SELECT COUNT(ld.id) AS device_count, ld.pid
FROM master_dev.legend_device ld
JOIN master_events.events_active ea
```

```
 ON ea.xid=ld.id WHERE ea.xtype=1 AND ea.etype=7052
GROUP BY pid;
```

- Given an alert_id (often found in the master_events.events_#### table entry for an event) produce the DA that goes with that alert.

```
select app_id from master.dynamic_app_alerts where alert_id=<alert_id>;
```

Aligned Dynamic Apps/Collectors/Powerpacks etc

Cleaning up unused PPKs and DAs

When Nightly Discovery is taking a long time or using too much CPU, it can be helpful to remove all the PPKs and/or DAs that aren't being used by a system. Unfortunately, the UI doens't have a way to display that so the following 2 queries were developed to help with that.

- List all DAs that are **NOT** aligned to a device for collection
  Written by @Ryan Peterson

```
SELECT master.dynamic_app.aid AS 'App ID'
FROM master.dynamic_app
LEFT JOIN master.dynamic_app_collection ON master.dynamic_app.aid =
master.dynamic_app_collection.app_id
WHERE master.dynamic_app_collection.app_id IS NULL;
```

- List all PPK ids and their names for any PPK whose device classes and DAs are **NOT** aligned to a device

```
SELECT DISTINCT(pp.ppid) as "PowerPack ID", pp.name as "PowerPack Name"
FROM master.powerpack pp
LEFT JOIN master.dynamic_app dap on (dap.ppguid = pp.ppguid)
LEFT JOIN master.dynamic_app_collection dac on (dac.app_id=dap.aid)
LEFT JOIN master.definitions_dev_classes dcl on (dcl.ppguid=pp.ppguid)
LEFT JOIN master.system_credentials sc on (sc.ppguid=pp.ppguid)
LEFT JOIN master.map_dynamic_app_device_cred mdadc on (mdadc.cred_id=sc.cred_id)
WHERE dap.ppguid IS NULL and dac.app_id IS NULL and mdadc.cred_id IS NULL
order by "PowerPack ID";
```

MariaDB Internals

- List 10 biggest tables in SQL

```
SELECT CONCAT(table_schema, '.', table_name) as tablename,
CONCAT(ROUND(table_rows / 1000000, 5), 'M') as num_rows,
CONCAT(ROUND(data_length / ( 1024 * 1024 * 1024 ), 2), 'G') as DATA,
CONCAT(ROUND(index_length / ( 1024 * 1024 * 1024 ), 2), 'G') as idx,
CONCAT(ROUND(( data_length + index_length ) / ( 1024 * 1024 * 1024 ), 2), 'G') as
total_size,
ROUND(index_length / data_length, 2) as idxfrac
FROM information_schema.TABLES
ORDER BY data_length + index_length DESC
LIMIT 10;
```

- For writing custom Snippet DAs, the following command will search DAs installed on the stack for snippets that include your keyword. It then gives you the app_id's it finds so you can review the code in the snippet.

```
select app_id
from master.dynamic_app_requests
where request like '%whatever_you_want_samples_of%'
```

Everything else

- Find scheduled actions in the past 30 days that took the most time (By `@Yaming Shao`)
  - the interval can be changed to suit your needs.
  - remove "and pa.policy_type="scheduled" to get all actions not just scheduled ones

```
SELECT pa.ap_id, aam.action_id, count(*) as actions_run, avg(aam.duration) as avg_time, max(aam.duration) as max_time
FROM master_logs.automation_action_metrics aam
INNER JOIN master.policies_automation pa ON aam.ap_id=pa.ap_id
WHERE aam.end_time > (NOW() - interval 30 day)
and pa.policy_type="scheduled"
GROUP BY aam.action_id
ORDER BY actions_run desc, avg_time desc
LIMIT 25;
```

- List all DAs for a given DID: Replace "<device_id>" with the did of the device

```
SELECT master.dynamic_app_collection.did as 'Device id', master_dev.legend_device.device as 'Device name', master.dynamic_app_collection.app_id as 'App ID', master.dynamic_app.name as 'App name'
FROM master.dynamic_app_collection
INNER JOIN master_dev.legend_device ON (master.dynamic_app_collection.did = master_dev.legend_device.id)
INNER JOIN master.dynamic_app ON (master.dynamic_app_collection.app_id = master.dynamic_app.aid)
WHERE master_dev.legend_device.id = <device_id>;
```

By changin the "where" above you can also get a list of all devices aligned to particular app id's.

```
SELECT master.dynamic_app_collection.did as 'Device id', master_dev.legend_device.device as 'Device name', master.dynamic_app_collection.app_id as 'App ID', master.dynamic_app.name as 'App name'
FROM master.dynamic_app_collection
INNER JOIN master_dev.legend_device ON (master.dynamic_app_collection.did = master_dev.legend_device.id)
INNER JOIN master.dynamic_app ON (master.dynamic_app_collection.app_id = master.dynamic_app.aid)
WHERE master.dynamic_app.aid = <APP_ID>
```

- List all collectors assigned to a given DID: replace “<device_id>” with the DID

```
silo_mysql -BNe "select ip from master.system_settings_licenses where function=5 and status=0"|xargs -I@ silo_mysql -h@ -P7707 -e "select (select id from master.system_settings_licenses) AS ApplianceID, did, critical_ping, avail_proto, avail_port from collector_state.V_device where did=<device_id>\G"
```

- List all devices of a given class and the AIDs associated with those devices: replace “<class_id>” with the id of the class you want to find

```
select ld.id, ld.device, ld.ip, dadc.app_id, da.name from master_dev.legend_device ld left join master.map_dynamic_app_device_cred dadc on (ld.id=dadc.did) left join master.dynamic_app da on (dadc.app_id=da.aid) where ld.class_type=<class_id>;
```

- For a given PPID list the devices with DAs from that PP aligned to them.

```
SELECT DISTINCT(ld.device), dst.state, ld.id, ld.ip, dcl.family, o.company, dcl.descript
FROM master_dev.legend_device ld
INNER JOIN master.dynamic_app_collection dac ON (dac.did=ld.id)
LEFT JOIN master.definitions_dev_classes dcl ON (ld.class_type=dcl.class_type)
LEFT JOIN master_biz.organizations o ON (ld.roa_id=o.roa_id)
LEFT JOIN master_dev.device_state dst ON (ld.id=dst.did)
LEFT JOIN master.dynamic_app dap ON (dac.app_id=dap.aid)
LEFT JOIN master.powerpack pp ON (dap.ppguid = pp.ppguid or dcl.ppguid=pp.ppguid)
WHERE ( pp.ppid=<ppid> );
```

- For a given DA ID (aid) list the devices it is aligned to.

```
SELECT DISTINCT(ld.device), dst.state, ld.id, ld.ip, dcl.family, o.company, dcl.descript
FROM master_dev.legend_device ld
INNER JOIN master.dynamic_app_collection dac ON (dac.did=ld.id)
LEFT JOIN master.definitions_dev_classes dcl ON (ld.class_type=dcl.class_type)
LEFT JOIN master_biz.organizations o ON (ld.roa_id=o.roa_id)
LEFT JOIN master_dev.device_state dst ON (ld.id=dst.did)
LEFT JOIN master.dynamic_app dap ON (dac.app_id=dap.aid)
WHERE ( dap.aid=<aid> );
```

- Find count of device schedules([provided here](#)):

```
        SELECT count(stt.schedule_id)
        FROM scheduler.schedules_to_tasks stt
        JOIN scheduler.schedules s
          ON stt.schedule_id = s.schedule_id
        JOIN scheduler.tasks t
          ON stt.task_id = t.task_id
         WHERE t.context in (2,5)
```

- Find devices discovered recently
  - Last Day - Counts of device class types and device class names

```sql
SELECT ddc.class, ddc.descript, ld.class_type, count(ld.class_type)
FROM master_dev.legend_device as ld
join master.definitions_dev_classes as ddc on ddc.id = ld.class_type
where create_date>= NOW() - INTERVAL 1 DAY  group by class_type;
```

- Last day - raw list

```sql
SELECT id, device, ip, create_date
FROM master_dev.legend_device
where create_date  >= NOW() - INTERVAL 1 DAY
order by create_date desc
```

- Last 12 hours

```sql
SELECT id, device, ip, create_date
FROM master_dev.legend_device
where create_date  >= NOW() - INTERVAL 12 HOUR
order by create_date desc
```

- Find all credentials that are from a PowerPack:

```sql
select sc.cred_id,sc.cred_name as Cred_Name,pp.name as PowerPack_Name
from master.system_credentials sc join master.powerpack pp on pp.ppguid = sc.ppguid
where sc.ppguid IS NOT null;
```

- Given an Event Policy ID, find out what PP it's from

```sql
SELECT *
FROM master.policies_events
WHERE id = <EVENT POLICY ID>;
```

- Find Dynamic Applications using an EE

```sql
SELECT da.name AS 'Dynamic Application', da.aid AS 'Dynamic Application ID', pp.name AS 'PowerPack', pp.ppid AS 'PowerPack ID'
FROM master.dynamic_app da
LEFT OUTER JOIN master.powerpack pp ON da.ppguid = pp.ppguid
LEFT OUTER JOIN master.exec_envs ee ON da.collection_env = ee.env_guid
WHERE ee.env_guid = '06AE66A017A5C6E1F2480C4FD462C63C'
ORDER BY pp.name
```

- Find RBA using an EE

```sql
SELECT pa.action_name AS 'Run Book Action', pa.action_id AS 'Run Book Action ID', pp.name AS 'PowerPack', pp.ppid AS 'PowerPack ID'
FROM master.policies_actions pa
LEFT OUTER JOIN master.powerpack pp ON pa.ppguid = pp.ppguid
LEFT OUTER JOIN master.exec_envs ee ON pa.collection_env = ee.env_guid
WHERE ee.env_guid = '06AE66A017A5C6E1F2480C4FD462C63C'
ORDER BY pp.name
```

Collector related

- Given a collector MID, output the device ids for all devices aligned to that collector:

```
select ld.id devices  from master_dev.legend_device ld
join master.definitions_dev_classes ddc on ld.class_type=ddc.id
where ld.pid = <mid>
```

- Given a collector MID, output the class name, class description, and count of all the devices with those classes for each device aligned to a collector. Replace <mid> with the collector ID.

```
select count(ld.id) as device_count, ld.class_type, ddc.class, ddc.descript
from master_dev.legend_device ld
join master.definitions_dev_classes ddc on ld.class_type=ddc.id
where ld.pid = <mid> group by class_type;
```

Execution Environment related

- List all EE's by name and guid (Written by @Dayn Harum)

```
select env_guid, env_name from master.exec_envs
 order by env_guid;
```

- list all EE's by guid, name, and associated PPK (Written by @Dayn Harum)

```
select distinct ee.env_guid, ee.env_name, ppack.name as ppack_name
  from master.exec_envs ee
left join master.dynamic_app dynapp
  on dynapp.collection_env = ee.env_guid
left join master.powerpack ppack
  on dynapp.ppguid = ppack.ppguid
 order by ee.env_guid;
```

Scheduler Related

- Given schedule ids, give all tasks raledted to those schedules as well as their last run time.

```
select ss.dtstart, ss.description, ss.schedule_id, st.task_id, st.name, st.xid,
st.enabled, st.last_run
from scheduler.schedules as ss
join scheduler.schedules_to_tasks as stt on ss.schedule_id= stt.schedule_id
join scheduler.tasks as st on stt.task_id = st.task_id
where ss.schedule_id in ('id1','id2')\G
```

General Usecases

Collecting info on Devices

- Query to capture all Gen1 monitored devices:

```
select count(*)
from master_dev.legend_device
where agent_type=4 and id not in (select device_id from insight_agent.common_agent)
```

- List all devices with agent_type 4

```sql
select id, device, agent_guid, agent_prime_backend
from master_dev.legend_device
where agent_type = 4;
```

- List all the devices stuck in the vanished table

```sql
SELECT  COUNT(*) FROM master_dev.legend_device_vanished ld
LEFT JOIN master_biz.organizations o ON (ld.roa_id = o.roa_id)
LEFT JOIN master_dev.device_state dst ON (dst.did = ld.id)
LEFT JOIN master.definitions_dev_classes dcl ON (ld.class_type = dcl.class_type)
LEFT JOIN master.definitions_dev_cats dca ON (dca.Fid=dcl.family)
INNER JOIN master_dev.component_dev_map cdm ON (cdm.component_did=ld.id)
LEFT JOIN master.system_credentials sc ON (sc.cred_id=ld.snmp_cred_id)
LEFT JOIN master.system_credentials_snmp scs ON (sc.cred_id=scs.cred_id)
INNER JOIN master.system_collector_groups scg ON (scg.cug_id = ld.cug_id)
LEFT JOIN (SELECT dtv.did,(vth+pth) AS vanish_and_purge_th, dtp.max AS purge_th
                        FROM (SELECT *, IFNULL(max,(SELECT component_vanish_timeout_mins
FROM master.system_settings_thresholds)) AS vth FROM master_dev.device_thresholds WHERE
sub_type= 3 AND type = 4 AND SUPER_TYPE = 0)dtv
                        INNER JOIN (SELECT *, IFNULL(max, (SELECT component_purge_timeout
FROM master.system_settings_thresholds)) AS pth FROM master_dev.device_thresholds WHERE
sub_type=2 AND type = 4 AND SUPER_TYPE = 0)dtp ON dtp.did = dtv.did) dt ON (dt.did =
ld.id)
WHERE 1 LIMIT 1
```

the full table data:

```sql
SELECT ld.device, dst.state, cdm.root_did, cdm.parent_did, ld.ip, ld.id, dca.cat_name,
dcl.class, dcl.descript, o.company, ld.cug_id, ld.active, UNIX_TIMESTAMP(ld.vanished_dt)
AS vanished_dt, IF(dt.purge_th = 0,"Never Purges",CAST(((dt.vanish_and_purge_th)-
((UNIX_TIMESTAMP(NOW())-UNIX_TIMESTAMP(vanished_dt))/3600)) AS UNSIGNED)) AS
hours_to_purge, ld.hours_till_purge
FROM master_dev.legend_device_vanished ld
LEFT JOIN master_biz.organizations o ON (ld.roa_id = o.roa_id)
LEFT JOIN master_dev.device_state dst ON (dst.did = ld.id)
LEFT JOIN master.definitions_dev_classes dcl ON (ld.class_type = dcl.class_type)
LEFT JOIN master.definitions_dev_cats dca ON (dca.Fid=dcl.family)
INNER JOIN master_dev.component_dev_map cdm ON (cdm.component_did=ld.id)
LEFT JOIN master.system_credentials sc ON (sc.cred_id=ld.snmp_cred_id)
LEFT JOIN master.system_credentials_snmp scs ON (sc.cred_id=scs.cred_id)
INNER JOIN master.system_collector_groups scg ON (scg.cug_id = ld.cug_id)
LEFT JOIN (SELECT dtv.did,(vth+pth) AS vanish_and_purge_th, dtp.max AS purge_th
            FROM (SELECT *, IFNULL(max,(SELECT component_vanish_timeout_mins FROM
master.system_settings_thresholds)) AS vth FROM master_dev.device_thresholds WHERE
sub_type= 3 AND type = 4 AND SUPER_TYPE = 0)dtv
            INNER JOIN (SELECT *, IFNULL(max, (SELECT component_purge_timeout FROM
master.system_settings_thresholds)) AS pth FROM master_dev.device_thresholds WHERE
sub_type=2 AND type = 4 AND SUPER_TYPE = 0)dtp ON dtp.did = dtv.did) dt ON (dt.did =
ld.id)
WHERE 1
ORDER BY device asc
```

- Turn on Debug logging for an SL1 process. Replace <AID> with the process ID for the process in question:

```
update master.system_settings_proc set debug=1 where aid=<AID>;
```

Searching MetaData (PowerPack and DA code, finding tables and columns in SQL, etc)

This section is for queries that get metadata in some form. Whether that is searching SQL itself to find out what tables has some bit of data, or searching DA's for code that they use. This will help navigate SQL for the unitiated, and/or provide useful examples for implementations

- Given a table name, find the database the table is in. Replace <table_name> with the table name you need to find the database for:

```
select table_schema as database_name
from information_schema.tables
where table_name = '<table_name>'
order by table_schema;
```

- Need an example of a Formula for a collection obj in a DA? This query spits out all the collection objects' formulas currently configured on the system. (Written by @Scott Karbiner and updated by @Jesse Triplett)

```
select formula from master.dynamic_app_objects where formula not like "" \G
```

- List all the OIDs that a Dynamic Application collects. Replace <DA ID> with the application ID of the DA in question @Danny Mazyn

```
select name, oid from master.dynamic_app_objects where app_id=<DA ID>
```

Miscellaneous

- Given a process id (found in the UI on System → Settings → Processes in the ID column and replacing <sp_id> below) and an appliance module ID (Found on System → Settings → Appliances page in the "id" column and replacing <mod_id>) Show the last 10 runs of a given process on a given appliance. Useful for seeing if it has completed recently.

```
SELECT start_time, end_time
FROM master_logs.system_runs
WHERE sp_id = <sp_id> and mod_id = <module_id> ORDER BY start_time DESC LIMIT 10;
```

Less Useful

Queries used by UI

Classic

- Number of Vanished Devices

```
SELECT  COUNT(*) FROM master_dev.legend_device_vanished ld
 LEFT JOIN master_biz.organizations o ON (ld.roa_id = o.roa_id)
 LEFT JOIN master_dev.device_state dst ON (dst.did = ld.id)
 LEFT JOIN master.definitions_dev_classes dcl ON (ld.class_type = dcl.class_type)
 LEFT JOIN master.definitions_dev_cats dca ON (dca.Fid=dcl.family)
 INNER JOIN master_dev.component_dev_map cdm ON (cdm.component_did=ld.id)
 LEFT JOIN master.system_credentials sc ON (sc.cred_id=ld.snmp_cred_id)
 LEFT JOIN master.system_credentials_snmp scs ON (sc.cred_id=scs.cred_id)
 INNER JOIN master.system_collector_groups scg ON (scg.cug_id = ld.cug_id)
 LEFT JOIN (SELECT dtv.did,(vth+pth) AS vanish_and_purge_th, dtp.max AS purge_th
                        FROM (SELECT *, IFNULL(max,(SELECT component_vanish_timeout_mins
FROM master.system_settings_thresholds)) AS vth FROM master_dev.device_thresholds WHERE
sub_type= 3 AND type = 4 AND SUPER_TYPE = 0)dtv
                        INNER JOIN (SELECT *, IFNULL(max, (SELECT component_purge_timeout
FROM master.system_settings_thresholds)) AS pth FROM master_dev.device_thresholds WHERE
sub_type=2 AND type = 4 AND SUPER_TYPE = 0)dtp ON dtp.did = dtv.did) dt ON (dt.did =
ld.id)
WHERE 1 LIMIT 1
```

- List first 500 vanished devices

```
SELECT ld.id AS vanished_dt, IF(dt.purge_th = 0,"Never
Purges",CAST(((dt.vanish_and_purge_th)-((UNIX_TIMESTAMP(NOW())-
UNIX_TIMESTAMP(vanished_dt))/3600)) AS UNSIGNED)) AS hours_to_purge, ld.hours_till_purge
FROM master_dev.legend_device_vanished ld
LEFT JOIN master_biz.organizations o ON (ld.roa_id = o.roa_id)
LEFT JOIN master_dev.device_state dst ON (dst.did = ld.id)
LEFT JOIN master.definitions_dev_classes dcl ON (ld.class_type = dcl.class_type)
LEFT JOIN master.definitions_dev_cats dca ON (dca.Fid=dcl.family)
INNER JOIN master_dev.component_dev_map cdm ON (cdm.component_did=ld.id)
LEFT JOIN master.system_credentials sc ON (sc.cred_id=ld.snmp_cred_id)
LEFT JOIN master.system_credentials_snmp scs ON (sc.cred_id=scs.cred_id)
INNER JOIN master.system_collector_groups scg ON (scg.cug_id = ld.cug_id)
LEFT JOIN (SELECT dtv.did,(vth+pth) AS vanish_and_purge_th, dtp.max AS purge_th
FROM (SELECT *, IFNULL(max,(SELECT component_vanish_timeout_mins FROM
master.system_settings_thresholds)) AS vth FROM master_dev.device_thresholds WHERE
sub_type= 3 AND type = 4 AND SUPER_TYPE = 0)dtv                        INNER JOIN
(SELECT *, IFNULL(max, (SELECT component_purge_timeout FROM
master.system_settings_thresholds)) AS pth FROM master_dev.device_thresholds WHERE
sub_type=2 AND type = 4 AND SUPER_TYPE = 0)dtp ON dtp.did = dtv.did) dt ON (dt.did =
ld.id) WHERE 1 ORDER BY device asc;
```

-

Other Userful Query Resources:

Pro Services: Useful SL1 Queries

Useful Commands (Non-sql)

These commands may make use of SQL but if so, they use regular bash, python, etc tools for processing the data in some way.

- Check collector status. Typically used to check if a DR node is ready to be failed over to.

```
/opt/em7/backend/collector_check.py
```

- Detailed DRBD Sync information *[NOTE]: May only work in a setup with DR as it leverages DRBD Proxy*

```
drbd-proxy-ctl -c "show hconnections";echo;drbd-proxy-ctl -c "show memusage"; echo; cat /proc/drbd;
```

- For rows behind it can be useful to know how many rows have been pulled. But the data in '/var/lib/em7/stats/collector' only outputs the net number of rows behind. So if those numbers are growing by 10k/minute, then that could mean that 20k rows come in and 10k get processed, or that 10k rows come in and no rows get processed on a minute to minute basis. To get more information about the rows pulled, we can run the silo_metrics script on one minute intervals like so:

```
/opt/em7/backend/silo_metrics.py mf_pull.pull.rows_pulled; sleep 60; /opt/em7/backend/silo_metrics.py mf_pull.pull.rows_pulled
```

  Additionally, you can swap "mf" for "lf" or "hf" if you need to focus on those frequencies of data.

Tidbits

Packet Captures on ESXI Hosts

At the time of this writing there doesn't appear to be a way to trigger a packet capture for an ESXI Host from the ui. You have to login. And you have to use `pktcap-uw` which does not have the flags such as `-c` that TCPDump has that allow you to continuously capture packets while rotating the output files to prevent filling up a filesystem.

On top of those challenges, ESXI hosts have only rudimentary `sh` shells not full `bash` shells and there is no `screen` installed which makes it more difficult to keep a capture running regardless of the session's connection status. And with the `sh` shell, this makes job control with things like fg, bg, jobs, and & difficult at best. Note in the script below how many times outputs have to be redirected to `/dev/null` for example.

At this time, the best option I have found is the following script from [this blog post](). However, it only captures for a short time and does not rotate files. **Use it with caution.**

```
#!/bin/sh
capture()
{
fileid=$1
pktcap-uw --uplink vmnic4 -o /vmfs/volumes/local-ds-86/vmnic4-inbound-file$fileid.pcap >/dev/null &
pktcap-uw --uplink vmnic4 --dir 1 -o /vmfs/volumes/local-ds-86/vmnic4-outbound-file$fileid.pcap >/dev/null &
pktcap-uw --uplink vmnic5 -o /vmfs/volumes/local-ds-86/vmnic5-inbound-file$fileid.pcap >/dev/null &
pktcap-uw --uplink vmnic5 --dir 1 -o /vmfs/volumes/local-ds-86/vmnic5-outbound-file$fileid.pcap >/dev/null &
}
#Change the number of iterations to reduce the total time of capture
for i in 0 1 2 3 4 5 6 7 8 9;
do
#Kill any existing session. This to make sure that the script can handle a previous dirty exit. For example a script exit using ctrl+c
kill -9 $(lsof | grep -i pktcap | grep vmfs | awk '{print $1}');
#Start captures
capture $i >/dev/null &
#Change the sleep time to reduce the time of capture per batch
sleep 900;
#Kill captures
kill -9 $(lsof | grep -i pktcap | grep vmfs | awk '{print $1}');
done
```

Pruner and Daily Maintenance

- Pruner info is stored in `master.system_settings_retention` and in that table in an `enabled` field which case be used to selectively turn on and off specific pruners. The names and IDs are:

ret_id	name
1	Availability Data
3	Avail Hourly Rollup
4	Avail Daily Rollup
5	CV Data
7	CV Hourly Rollup
8	CV Daily Rollup
9	DNS Data

11	DNS Hourly Rollup
12	DNS Daily Rollup
13	RT Email Data
15	RT Email Hourly Rollup
16	RT Email Daily Rollup
17	Port Data
19	Port Hourly Rollup
20	Port Daily Rollup
21	Process Data
23	Process Hourly Rollup
24	Process Daily Rollup
25	Service Data
27	Service Hourly Rollup
28	Service Daily Rollup
29	TV Data
31	TV Hourly Rollup
32	TV Daily Rollup
33	Access Failed Logs
34	Access Logs
35	Discovery Logs
36	Notifier Logs
37	Process Spool Logs
38	System Logs
39	Process Run Log
40	Automation Result Log

42	Queued Automation Policies
43	Network Logs
44	Scheduled Report Files
45	Exec Report Files
46	Cleared Events
47	Pending Events
50	Exception Log
51	Spooled Email
52	Interface Data
54	Interface Hourly Rollup
55	Interface Daily Rollup
56	Virtual Interface Data
57	Virtual Interface Hourly Rollup
58	Virtual Interface Daily Rollup
59	Configuration Dynamic App Data
60	Performance Dynamic App Data
62	Normalized (H) Dynamic App Data
63	Normalized (D) Dynamic App Data
65	Dynamic App Journal
68	Device Logs (By age)
69	Device Logs (By size)
70	Outage Table: Avail
71	Outage Table: Port
72	Outage Table: CV
73	Outage Table: TV

74	Outage Table: DNS
75	Outage Table: Service
76	Outage Table: Process
77	Outage Table: Email
78	Outage Table: Storage
79	Automation Logs
80	ITSM Raw Data Points
81	ITSM Service Metrics
83	Normalized (H) ITSM Service Metr
84	Normalized (D) ITSM Service Metr
85	ITSM Key Metrics
87	Normalized (H) ITSM Key Metrics
88	Normalized (D) ITSM Key Metrics
89	Normalizer Log
90	Storage Data
92	Storage Hourly Rollup
93	Storage Daily Rollup
94	Dev Usage Stats Rollup Hourly
95	Dev Usage Stats Rollup Daily
96	Retention Log
97	Interface Perc Util Data
99	Interface Perc Util Hourly Rollu
100	Interface Perc Util Daily Rollup
101	Device Ratings Hourly
102	Device Ratings Rollup Daily

103	System Usage Counters Hourly
104	System Usage Hourly
105	Audit Logs
106	Subscriber Device Configuration
107	Subscriber Device Type Data
108	Subscriber System Configuration
109	Subscriber System Usage Data
110	Subscriber Daily Delivery Data
111	Normalized (H) Vitals Data
112	Normalized (D) Vitals Data
113	Subscriber Device Usage Data
114	Toolbox Remote Logs
115	Toolbox Device Logs
116	CBQoS Performance Data
118	Normalized (H) CBQoS Performance
119	Normalized (D) CBQoS Performance
120	Interface Extended Data
122	Interface Extended Hourly Rollup
123	Interface Extended Daily Rollup
124	Automation Action Metrics
125	Automation Policy Metrics
126	Normalized (H) HAR Hourly Rollup
127	Normalized (D) HAR Daily Rollup
128	HAR Key Data

129	HAR Individual Table Data
130	HAR Process Log Table Data
131	HAR Master Process Log Table Dat
132	HAR RCA Data
133	Silo Metrics: Dynamic App Jobs
134	Silo Metrics: Interface Collecti
135	Streamer Push Availability
136	Streamer Push Interface
137	Streamer Push DA Perf/Conf
138	Schedules
139	RBA Queue Size
140	Raw Event Insights Alert Count
141	Raw Event Insights Event Metrics
142	Base Event Insights Alert counts
143	Base Event Insights Event metric
144	Base Event Insights Device count
145	Hourly Event Insights Alert coun
146	Hourly Event Insights Event metr
147	Hourly Event Insights Device cou
148	DAILY Event Insights Alert count
149	DAILY Event Insights Event metri
150	DAILY Event Insights Device coun

Daily Maint and Pruning resources:

Data Pruner is not Completing

Pruner documentation